

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №2
Тема: Основи проектування

Виконав:
студент групи ІА-34
Єресько І.С
Мягкий М.Ю

Київ-2025

Тема: Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server) Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

Мета: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проєктується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області.

1.Короткі теоретичні відомості

У даній роботі було виконано проєктування системи PowerShell Terminal з використанням UML-діаграм та моделювання її функціональності та структури. Основні етапи роботи:

1. Діаграми варіантів використання (Use Case Diagram)

Використано для візуалізації взаємодії користувача з системою.

Визначено основних акторів (Користувач) та їх прецеденти:

Виконання команд та скриптів, управління інтерфейсом та вкладками, збереження результатів, робота з історією команд.

Для кожного прецеденту описані сценарії виконання, передумови, постумови та можливі винятки.

2. Діаграма класів (Class Diagram)

Відображає структуру системи та взаємозв'язки між основними класами:

Main Application: MainForm

Managers: TabManager, ThemeManager, HistoryService

Repository Layer: CommandHistoryRepository,
UserSettingsRepository, TerminalSessionRepository

Domain Models: TerminalTab, TerminalControl,
CommandInterpreter, TerminalTheme

Database Entities: CommandHistory, UserSettings,
TerminalSession, TerminalDbContext

3. Прецеденти

Детально описані основні сценарії використання системи:

Виконання команди PowerShell

Створення нової вкладки

Зміна теми інтерфейсу

4. Система бази даних

Для реалізації обрана SQL Server Management Studio (SSMS).

База даних PowerShellTerminalDB зберігає інформацію про сесії, історію команд та налаштування користувачів.

Таблиці:

UserSettings – налаштування користувача (шрифти, кольори, тема, розміри вікна).

TerminalSessions – інформація про сесії терміналу.

CommandHistories – історія виконаних команд із результатами.

2. Use Case Diagram

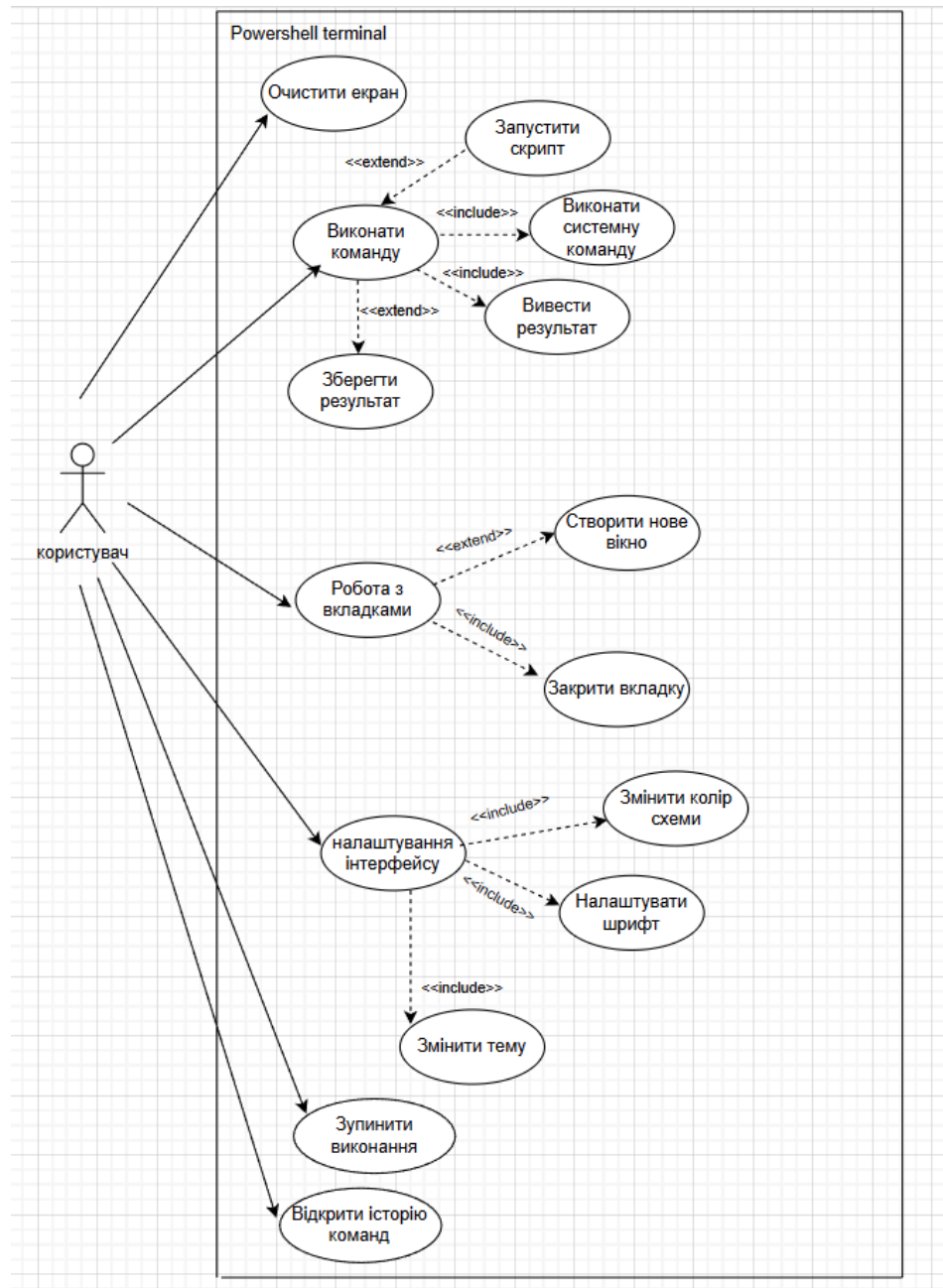


Рис 2.1: Use Case Diagram

Опис Use Case діаграми

Актор:

Користувач - взаємодіє з PowerShell Terminal для виконання команд, налаштування інтерфейсу та керування сесіями.

Основні варіанти використання:

1. Очистити екран

Опис: користувач очищає вікно терміналу від попередніх результатів.

Результат: екран порожній, готовий до нових команд

2. Запустити скрипт (extends “Виконати команду”)

Опис: користувач запускає заздалегідь підготовлений PowerShell-скрипт.

Розширює: сценарій “Виконати команду”.

3. Виконати команду

Опис: основна функція терміналу - виконання введених користувачем команд.

Включає:

“Виконати системну команду”

“Вивести результат”

Розширює:

“Зберегти результат”.

4. Виконати системну команду (include)

Опис: виконання команди, що взаємодіє з ОС (наприклад, робота з файлами чи процесами).

5. Вивести результат (include)

Опис: після виконання команди система відображає результат у терміналі.

6. Зберегти результат (extend)

Опис: користувач може зберегти вихідні дані виконаної команди у файл.

Керування інтерфейсом і вкладками:

7. Робота з вкладками

Опис: користувач може працювати в кількох вкладках.

Включає: “Закрити вкладку”.

Розширює: “Створити нове вікно”.

8. Закрити вкладку (include)

Опис: закриття активної вкладки без завершення всієї сесії.

9. Створити нове вікно (extend)

Опис: відкриття нового вікна PowerShell для окремої сесії.

Налаштування інтерфейсу:

10. Налаштування інтерфейсу

Опис: користувач змінює зовнішній вигляд терміналу.

Включає:

“Змінити колір схеми”

“Налаштувати шрифт”

“Змінити тему”.

11.Змінити колір схеми (include)

Опис: вибір кольорової палітри для інтерфейсу.

12.Налаштувати шрифт (include)

Опис: зміна розміру та типу шрифту в консолі.

13.Змінити тему (include)

Опис: вибір між світлою або темною темою.

14.Зупинити виконання

Опис: користувач перериває поточний процес або скрипт.

Відкрити історію команд

Опис: перегляд і повторне виконання раніше введених команд.

3 Діаграма класів

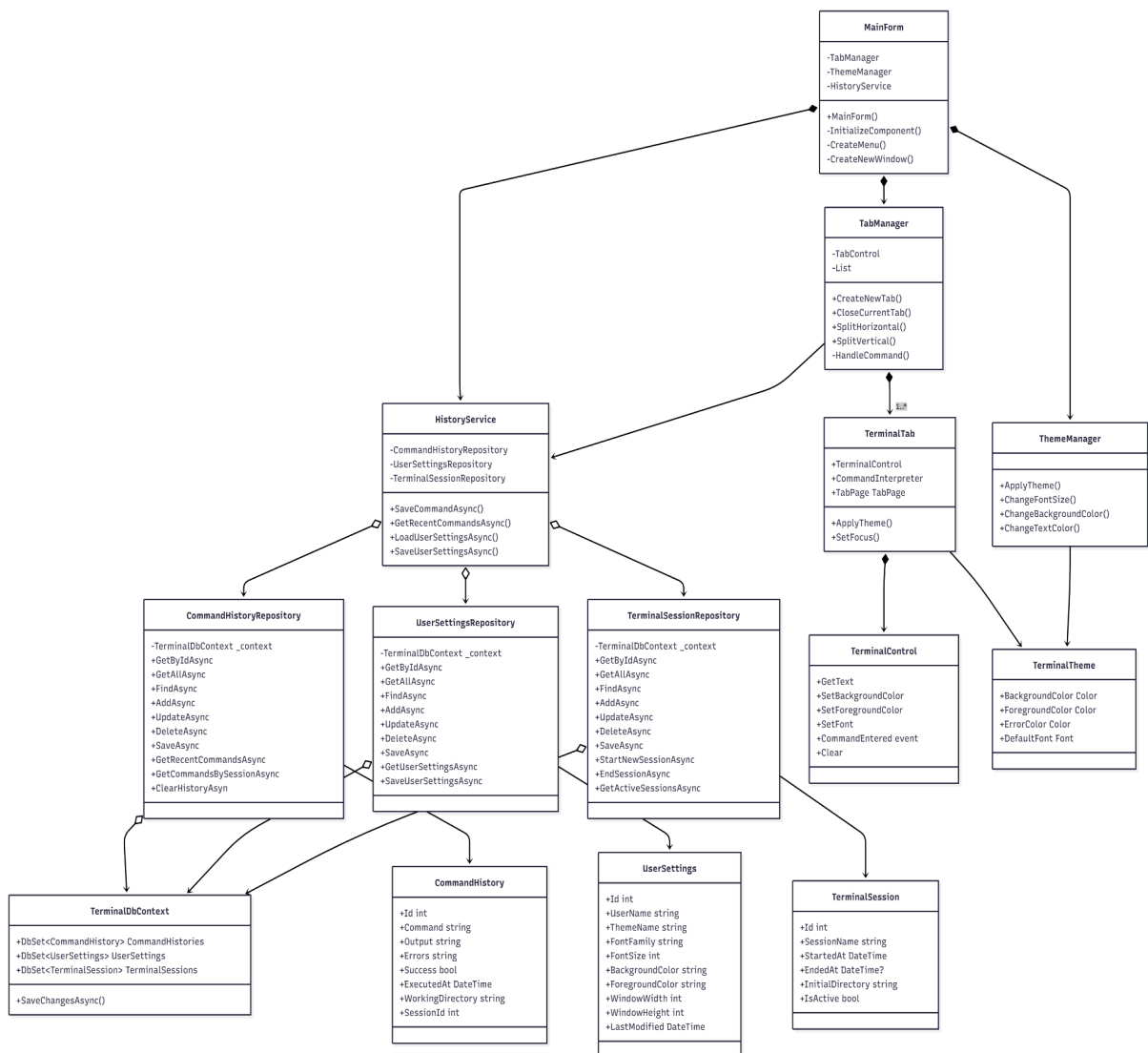


Рис 3.1: Діаграма класів (Class Diagram)

Головна форма

- MainForm - головне вікно програми, координує всі компоненти

Менеджер

- TabManager - керує вкладками (створення, закриття, розділення)

- ThemeManager - керує темами, кольорами, шрифтами

Сервіси та репозиторії

- HistoryService- зберігає/завантажує історію команд та налаштування
- CommandHistoryRepository - робота з історією команд в БД
- UserSettingsRepository - робота з налаштуваннями користувача
- TerminalSessionRepository - робота з сесіями терміналу

Компоненти терміналу

- TerminalTab - окрема вкладка терміналу
- TerminalControl - власне поле введення/виводу команд
- CommandInterpreter - інтерпретатор команд (виконує команди)
- TerminalTheme - налаштування кольорів та шрифтів

Сутності бази даних

- CommandHistory - запис про виконану команду
- UserSettings - налаштування користувача
- TerminalSession - сесія роботи з терміналом
- TerminalDbContext - підключення до бази даних

Шаблон Repository

Окремі класи для кожної сутності (CommandHistoryRepository, UserSettingsRepository, TerminalSessionRepository)

Інкапсуляція доступу до БД - вся робота з базою через репозиторії
CRUD методи - GetById, GetAll, Add, Update, Delete

Спеціалізовані методи - GetRecentCommandsAsync, GetUserSettingsAsync тощо

Потік роботи

1. MainForm запускає TabManage для створення вкладок
2. Кожна TerminalTab містить TerminalControl для вводу команд
3. CommandInterpreter виконує команди
4. HistoryService зберігає результати через репозиторії в БД
5. ThemeManager застосовує вигляди до всіх компонентів

3.Прецеденти

Прецедент 1: Виконання PowerShell команди

Актор: Користувач

Ціль: Виконати команду PowerShell та отримати результат

Передумови: Термінал запущено

Основний сценарій:

- Користувач вводить команду у поле вводу
- Система передає команду інтерпретатору
- Інтерпретатор вибирає стратегію виконання
- Виконується команда через PowerShell
- Результат виводиться у термінал

Постумови: Команда виконана, результат відображено

Вийнятки:

1а. Команда введена з помилкою - система виводить повідомлення про синтаксичну помилку

1б. Відсутні права доступу - система повідомляє про відмову у доступі

1в. PowerShell команди не існує - система показує повідомлення про неможливість виконання

Прецедент 2: Створення нової вкладки

Актор: Користувач

Ціль: Створити нову сесію терміналу у вкладці

Передумови: Головне вікно відкрите

- Основний сценарій:
- Користувач обирає "File - New Tab"
 - Система створює нову вкладку
 - Застосовується поточна тема
 - Нова вкладка стає активною

Постумови: Створено нову вкладку терміналу

Вийнятки:

- 2а. Недостатньо пам'яті для створення нової вкладки - система виводить повідомлення про помилку
- 2б. Помилка ініціалізації контролю терміналу - вкладка не створюється, система повідомляє користувача
- 2в. Система повертає null - система скасовує створення вкладки

Прецедент 3: Зміна теми інтерфейсу

Актор: Користувач

Ціль: Змінити колірну схему терміналу

Передумови: Термінал запущено

Основний сценарій:

- Користувач обирає "View - Dark Theme"
- Система створює нову тему
- Застосовує тему до всіх відкритих вкладок
- Оновлює кольори тексту та фону

Постумови: Тема змінена для всіх вкладок

Вийнятки:

- 3а. Обрана тема відсутня або пошкоджена - система повертає стандартну тему
- 3б. Помилка оновлення стилів - система частково оновлює інтерфейс і повідомляє користувача
- 3в. Відсутній доступ до файлів теми - система показує повідомлення про неможливість зміни теми

Постумови: Тема змінена для всіх вкладок

4. Структура системи та основні класи

// Main Application

[public class MainForm : Form](#)

// Managers

[public class TabManager](#)

[public class ThemeManager](#)

[public class HistoryService](#)

// Repository Layer

[public class CommandHistoryRepository](#)

[public class UserSettingsRepository](#)

[public class TerminalSessionRepository](#)

// Core Models

[public class TerminalTab](#)

[public class TerminalControl](#)

[public class CommandInterpreter { }](#)

[public class TerminalTheme](#)

// Database Entities

[public class CommandHistory](#)

[public class UserSettings](#)

[public class TerminalSession](#)

[public class TerminalDbContext](#)

5. Система баз даних.

Для реалізації цієї бази даних було вирішено використовувати SQL Server Management Studio (SSMS). Це дозволяє ефективно створювати та адмініструвати таблиці, встановлювати зв'язки між ними, виконувати SQL-запити та керувати даними у структурованому середовищі. Використання SSMS забезпечує зручний графічний інтерфейс для розробки бази даних та управління сесіями і історією команд.

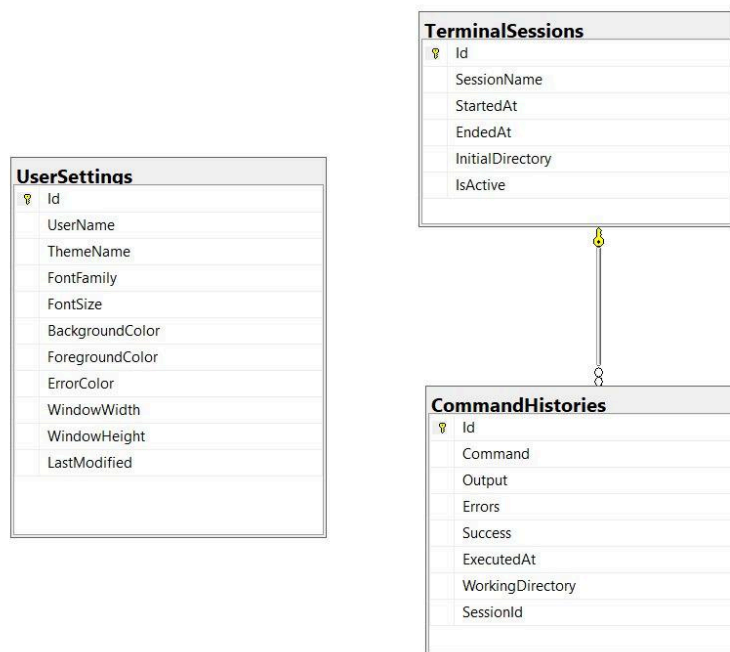


Рис 5.1:База даних (PowerShellTerminalDB)

Опис бази даних

Назва БД: (PowerShellTerminalDB)

Мета: Зберігання даних про сесії терміналу, історію команд та налаштування користувачів.

Таблиці

1. UserSettings

Зберігає налаштування користувачів терміналу.

Поля:

- Id - унікальний ідентифікатор користувача.
- UserName - ім'я користувача.
- ThemeName - назва теми оформлення.
- FontFamily - шрифт терміналу.
- FontSize - розмір шрифту.
- BackgroundColor - колір фону.
- ForegroundColor - колір тексту.
- ErrorColor - колір для повідомлень про помилки.

- WindowWidth - ширина вікна терміналу.
- WindowHeight - висота вікна терміналу.
- LastModified - дата останньої зміни налаштувань.

2. TerminalSessions

Зберігає інформацію про сесії терміналу.

Поля:

- Id - унікальний ідентифікатор сесії.
- SessionName - назва сесії.
- StartedAt - дата та час початку сесії.
- EndedAt - дата та час завершення сесії.
- InitialDirectory - початковий каталог терміналу.
- IsActive - прапорець активності сесії (true/false).

3. CommandHistories

Зберігає історію виконаних команд у сесіях терміналу.

Поля:

- Id - унікальний ідентифікатор запису.
- Command - виконана команда.
- Output - результат виконання команди.
- Errors - помилки виконання (якщо були).
- Success - прапорець успішності виконання.
- ExecutedAt - дата та час виконання команди.
- WorkingDirectory - робочий каталог під час виконання.
- SessionId - зв'язок із таблицею TerminalSessions.

Взаємозв'язки

TerminalSessions з CommandHistories: один-до-багатьох (1:N).

Кожна сесія може містити багато записів історії команд.

6. Контрольні запитання

1. **UML (Unified Modeling Language)** – стандартизована мова для візуального моделювання програмних систем, яка дозволяє описувати структуру, поведінку та взаємодію елементів системи.
2. **Діаграма класів UML** – графічне відображення класів системи, їхніх атрибутів, методів та зв'язків між ними.
3. **Канонічні діаграми UML** – основні типи:

Діаграми класів

Діаграми об'єктів

Діаграми варіантів використання (use case)

Діаграми послідовності

Діаграми станів

4. **Діаграма варіантів використання (use case diagram)** – показує варіанти використання та акторів, які взаємодіють із системою.
5. **Варіант використання** – конкретна функція або процес системи, яку користувач (актор) може виконати для досягнення певної мети.
6. **Відношення на діаграмі використання:**

Асоціація (association) – узагальнене, невідоме ставлення між актором та варіантом використання.

Відношення узагальнення (generalization) – показує, що нащадок успадковує атрибути у свого прямого батьківського елемента.

Відношення залежності (dependency) визначається як форма взаємозв'язку між двома елементами моделі, призначена для специфікації тієї обставини, що зміна одного елемента моделі призводить до зміни деякого іншого елемента. 24

Відношення включення (include) – окремий випадок загального відношення залежності між двома варіантами використання, при якому деякий варіант використання містить поведінку, визначену в іншому варіанті використання.

Відношення розширення (extend) – показує, що варіант використання розширює базову послідовність дій та вставляє власну послідовність.

7. **Сценарій** – опис конкретного виконання варіанту використання у вигляді послідовності кроків.

8. **Діаграма класів** – статичне представлення структури системи, яке показує класи, їх атрибути, методи та зв'язки між класами.

9. **Зв'язки між класами:**

Асоціація

Узагальнення (наслідування)

Агрегація

Композиція

Реалізація (інтерфейси)

10. **Відмінність композиції від агрегації:**

Агрегація – слабкий «has-a» зв'язок, частина може існувати окремо від цілого.

Композиція – сильний «частина-ціле» зв'язок, частина не може існувати без цілого.

11. **На діаграмах класів:**

Агрегація позначається порожнім ромбом на стороні цілого.

Композиція позначається заповненим ромбом на стороні цілого.

12. Нормальні форми баз даних – правила організації даних у таблицях, які зменшують надлишковість і запобігають аномаліям при оновленні даних.

13. Фізична модель БД – опис структури таблиць, полів, індексів, ключів та зв'язків у конкретній СУБД.

Логічна модель БД – концептуальне представлення даних, їх атрибутів і зв'язків незалежно від конкретної СУБД.

14. Взаємозв'язок між таблицями БД та класами – таблиці відповідають класам, поля таблиць – атрибутам класів, зовнішні ключі – асоціаціям між класами.

Висновок:

У ході роботи було обрано зручну систему для побудови UML-діаграм, опановано основи створення діаграм варіантів використання для проєктованої системи, розроблено сценарії варіантів використання та побудовано діаграму класів предметної області. Отримані навички дозволяють ефективно моделювати структуру та поведінку програмних систем, що є важливим етапом у процесі їх проєктування.